Lecture 16: Power iteration

Jamie Haddock

Table of contents

1 Power iteration **1**
- 1.1 Demo . . . 1
- 1.2 Dominant eigenvalue . . . 2
- 1.3 Power iteration . . . 3
- 1.4 Convergence . . . 4

2 Inverse iteration **7**
- 2.1 Inverse iteration . . . 8
- 2.2 Algorithm . . . 8
- 2.3 Convergence . . . 9
- 2.4 Dynamic shifting . . . 11

1 Power iteration

As mentioned previously real-world data is often sparse, and matrix-vector multiplication is fast for sparse matrices, so this operation is quite efficient. Methods based upon matrix-vector multiplications can be very efficient! We'll see the most famous such algorithm in this lecture.

1.1 Demo

```
%%julia
# choose a random 5×5 matrix and a random 5-vector
A = rand(1.:9.,5,5)
A = A./sum(A,dims=1)
x = randn(5);
```

A couple of matrix multiplications doesn't reveal anything…

```
%%julia
y = A*x
z = A*y
```

```
5-element Vector{Float64}:
 -0.07267133409426654
 -0.09325440634931677
 -0.08863636523365853
 -0.07129733191997407
 -0.09047120737644238
```

But many matrix multiplications do! Eventually $\mathbf{Ax} \approx \mathbf{x}$!

```
%%julia
for j in 1:8;  x = A*x;  end
[x A*x]
```

```
5×2 Matrix{Float64}:
 -0.0614713  -0.0614688
 -0.116411   -0.116415
 -0.109326   -0.10933
 -0.0567551  -0.0567538
 -0.0723672  -0.0723634
```

This phenomenon seems to occur regardless of the starting vector.

```
%%julia
x = randn(5)
for j in 1:8;  x = A*x;  end
[x A*x]
```

```
5×2 Matrix{Float64}:
 -0.329006  -0.329003
 -0.623085  -0.623091
 -0.585161  -0.585168
 -0.303769  -0.303767
 -0.387323  -0.387317
```

There was a little cheating in this demo to make the story come out neatly (specifically, the line `A=A./sum(A,dims=1)`). But it illustrates an important general fact that we investigate now.

1.2 Dominant eigenvalue

Analysis of matrix powers is most straightforward in the diagonalizable case. Let $\mathbf{A}$ be any diagonalizable $n \times n$ matrix having eigenvalues $\lambda_1, \dots, \lambda_n$ and corresponding linearly independent eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_n$.

Furthermore, suppose the eigenvalues are such that

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|.$$

Given such an ordering we say that λ_1 is the **dominant eigenvalue**. This was the case with $\lambda_1 = 1$ for $\mathbf{A}$ in the previous demo.

Now let $\mathbf{x}$ be an n-vector, let k be a positive integer, and consider:

$$\mathbf{A}^k\mathbf{x} = \mathbf{V}\mathbf{D}^k\mathbf{V}^{-1}\mathbf{x}.$$

Let $\mathbf{z} = \mathbf{V}^{-1}\mathbf{x}$, and recall that $\mathbf{D}$ is a diagonal matrix of eigenvalues. Then

$$\begin{aligned}\mathbf{A}^k\mathbf{x} = \mathbf{V}\mathbf{D}^k\mathbf{z} &= \mathbf{V}\begin{bmatrix}\lambda_1^k z_1\\ \lambda_2^k z_2\\ \vdots\\ \lambda_n^k z_n\end{bmatrix}\\ &= \lambda_1^k\left[z_1\mathbf{v}_1 + z_2\left(\frac{\lambda_2}{\lambda_1}\right)^k\mathbf{v}_2 + \cdots + z_n\left(\frac{\lambda_n}{\lambda_1}\right)^k\mathbf{v}_n\right].\end{aligned}$$

Since λ_1 is dominant, we conclude that if $z_1 \neq 0$,

$$\left\| \frac{\mathbf{A}^k \mathbf{x}}{\lambda_1^k} - z_1 \mathbf{v}_1 \right\| \leq |z_2| \cdot \left| \frac{\lambda_2}{\lambda_1} \right|^k \|\mathbf{v}_2\| + \cdots + |z_n| \cdot \left| \frac{\lambda_n}{\lambda_1} \right|^k \|\mathbf{v}_n\| \to 0 \text{ as } k \to \infty.$$

That is, $\mathbf{A}^k \mathbf{x}$ eventually is a scalar multiple of the dominant eigenvector as long as $z_i \neq 0$. If $\mathbf{x}$ is chosen randomly, the probability that $z_1 = 0$ is mathematically zero.

For algorithmic purposes, it is important to interpret $\mathbf{A}^k \mathbf{x}$ as $\mathbf{A}(\cdots(\mathbf{A}(\mathbf{A}\mathbf{x}))\cdots)$, i.e., as repeated applications of $\mathbf{A}$ to a vector. Doing so allows us to fully exploit sparsity of $\mathbf{A}$, something which is not preserved by taking a matrix power $\mathbf{A}^k$ explicitly before the multiplication with $\mathbf{x}$.

1.3 Power iteration

We don't quite yet have an algorithm.

Unless $|\lambda_1| = 1$, the factor λ_1^k tends to make $\|\mathbf{A}^k \mathbf{x}\|$ either very large or very small. Nor can we easily normalize by λ_1^k, unless we know λ_1 in advance.

To make a practical algorithm, we alternate matrix-vector multiplication with a renormalization of the vector. In the following, we use $x_{k,m}$ and $y_{k,m}$ to mean the mth component of vectors $\mathbf{x}_k$ and $\mathbf{y}_k$.

Algorithm: Power iteration

Given matrix $\mathbf{A}$:

1. Choose $\mathbf{x}_1$.
2. For $k = 1, 2, ...,$
   a. Set $\mathbf{y}_k = \mathbf{A}\mathbf{x}_k$.
   b. Find m such that $|y_{k,m}| = \|\mathbf{y}_k\|_\infty$.
   c. Set $\alpha_k = \dfrac{1}{y_{k,m}}$ and $\beta_k = \dfrac{y_{k,m}}{x_{k,m}}$.
   d. Set $\mathbf{x}_{k+1} = \alpha_k \mathbf{y}_k$.

Note that by definition, $\|\mathbf{x}_{k+1}\|_\infty = 1$. Also, we can write

$$\mathbf{x}_k = (\alpha_1 \alpha_2 \cdots \alpha_k) \mathbf{A}^k \mathbf{x}_1.$$

Finally, if $\mathbf{x}_k$ is nearly a dominant eigenvector of $\mathbf{A}$, then $\mathbf{A}\mathbf{x}_k$ is nearly $\lambda_1 \mathbf{x}_k$, and we can take the ratio $\beta_k = y_{k,m}/x_{k,m}$ as an eigenvalue estimate.

In fact, the extra α_j normalization factors cancel in the ratio, and, after some simplification, we get

$$\beta_k = \frac{y_{k,m}}{x_{k,m}} = \lambda_1 \frac{1 + r_2^{k+1} b_2 + \cdots + r_n^{k+1} b_n}{1 + r_2^k b_2 + \cdots + r_n^k b_n},$$

where $r_j = \lambda_j / \lambda_1$ and the b_j are constants.

By the assumed ording of the eigenvalues, each r_j satisfies $|r_j| < 1$, so we see that $\beta_k \to \lambda_1$ as $k \to \infty$.

```
%%julia
"""
    poweriter(A,numiter)

Perform `numiter` power iterations with the matrix `A`, starting
from a random vector. Returns a vector of eigenvalue estimates
and the final eigenvector approximation.
"""
function poweriter(A,numiter)
    n = size(A,1)
    x = normalize(randn(n),Inf)
      = zeros(numiter)
    for k in 1:numiter
        y = A*x
        m = argmax(abs.(y))
         [k] = y[m]/x[m]
        x = y/y[m]
    end
    return  ,x
end
```

```
poweriter (generic function with 1 method)
```

Observe that the only use of $\mathbf{A}$ is to find the matrix-vector product $\mathbf{Ax}$, which easily exploits the sparsity of $\mathbf{A}$.

1.4 Convergence

Let's examine the terms in the numerator and denominator of

$$\beta_k = \frac{y_{k,m}}{x_{k,m}} = \lambda_1 \frac{1 + r_2^{k+1} b_2 + \cdots + r_n^{k+1} b_n}{1 + r_2^k b_2 + \cdots + r_n^k b_n},$$

more carefully:

$$\begin{aligned} r_2^k b_2 + \cdots + r_n^k b_n &= r_2^k \left[b_2 + \left(\frac{r_3}{r_2}\right)^k b_3 + \cdots + \left(\frac{r_n}{r_2}\right)^k b_n \right] \\ &= r_2^k \left[b_2 + \left(\frac{\lambda_3}{\lambda_2}\right)^k b_3 + \cdots + \left(\frac{\lambda_n}{\lambda_2}\right)^k b_n \right]. \end{aligned}$$

At this point we'll introduce an additional assumption,

$$|\lambda_2| > |\lambda_3| \geq \cdots \geq |\lambda_n|.$$

This isn't srictly necessary, but it makes the analysis easier.

With this assumption, we see

$$r_2^k b_2 + \cdots + r_n^k b_n \to b_2 r_2^k \text{ as } k \to \infty.$$

Using a geometric series expansion for the denominator (and ignoring terms of order $2k$), we have

$$\begin{aligned} \beta_k &\to \lambda_1 \left(1 + b_2 r_2^{k+1}\right) \left(1 - b_2 r_2^k + O(r_2^{2k})\right), \\ \beta_k - \lambda_1 &\to \lambda_1 b_2 (r_2 - 1) r_2^k. \end{aligned}$$

This is linear convergence with factor r_2:

$$\frac{\beta_{k+1} - \lambda_1}{\beta_k - \lambda_1} \to r_2 = \frac{\lambda_2}{\lambda_1} \quad \text{as } k \to \infty.$$

The error in the power iteration eigenvalue estimates β_k is reduced asymptotically by a constant factor λ_2/λ_1 at each iteration.

We will experiment with the power iteration on a 5×5 matrix with prescribed eigenvalues and dominant eigenvalue at 1.

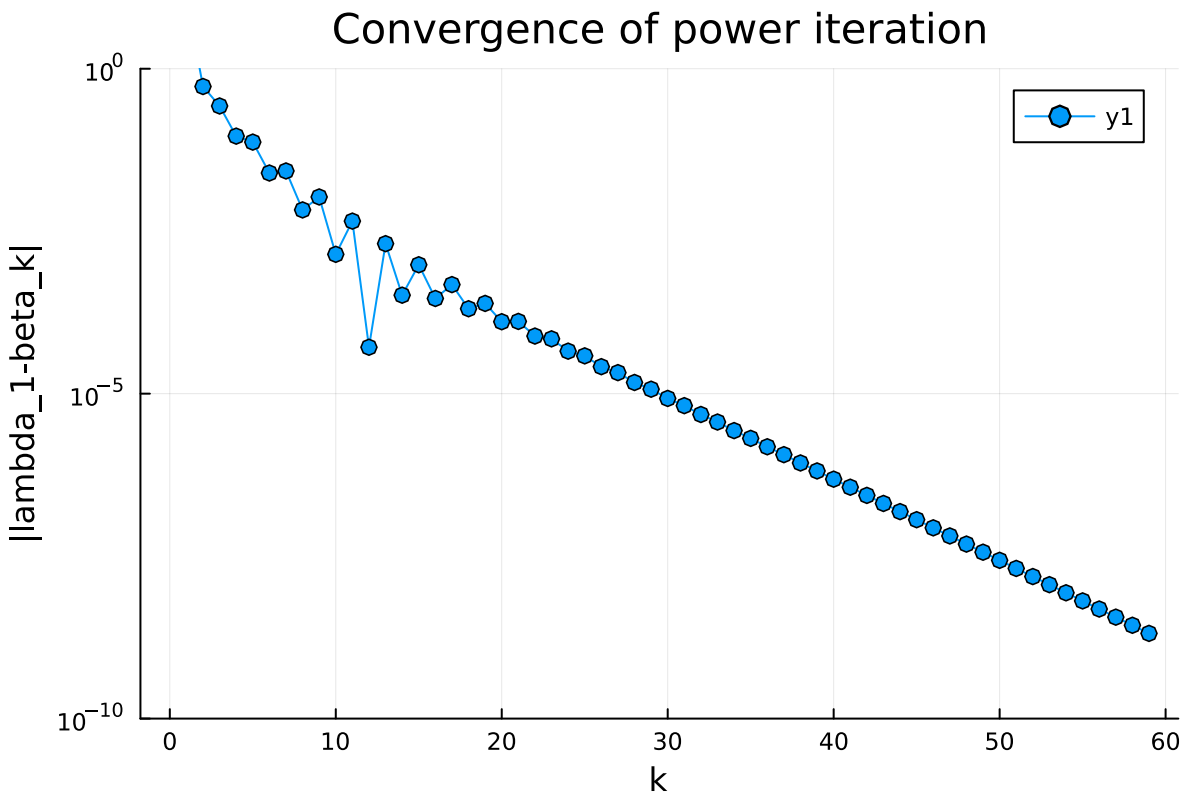
```
%%julia
  = [1,-0.75,0.6,-0.4,0]
# Make a triangular matrix with eigenvalues on the diagonal.
A = triu(ones(5,5),1) + diagm( )
```

```
5×5 Matrix{Float64}:
 1.0   1.0   1.0   1.0  1.0
 0.0  -0.75  1.0   1.0  1.0
 0.0   0.0   0.6   1.0  1.0
 0.0   0.0   0.0  -0.4  1.0
 0.0   0.0   0.0   0.0  0.0
```

```
%%julia
 ,x = poweriter(A,60)
eigval =  [end]
```

```
1.0000000020466426
```

```
%%julia
errs = @. 1 -
plot(0:59,abs.(errs),m=:o,title="Convergence of power iteration",
    xlabel="k",yaxis=("|lambda_1-beta_k|",:log10,[1e-10,1]))
```



The asymptotic trend seems to be a straight line, consistent with linear convergence. To estimate the convergence rate, we look at the ratio of two consecutive errors in the linear part of the convergence curve. This should match the ratio of the first two eigenvalues.

```
%%julia
@show theory =  [2]/ [1];
@show observed = errs[40]/errs[39];
```

```
theory =  [2] /  [1] = -0.75
observed = errs[40] / errs[39] = -0.7539539990178831
```

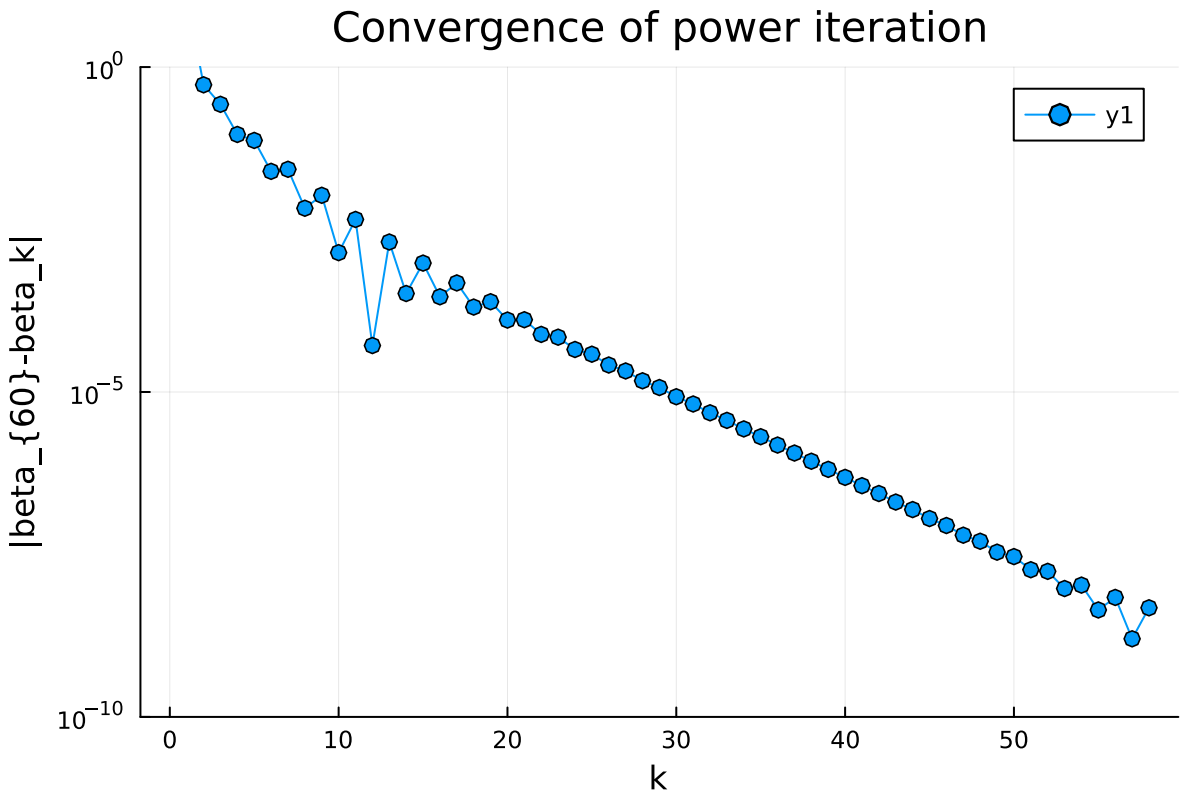
Note that the error is supposed to change sign on each iteration. The effect of these alternating signs is that estimates oscillate around the exact value.

```
%%julia
 [26:30]
```

```
5-element Vector{Float64}:
 1.0000381445886617
 0.9999739896237696
 1.000021066514551
 0.999985135362209
 1.0000117096015708
```

In practical situations, we don't know the exact eigenvalue that the algorithm is supposed to find. In that case we would base errors on the final β that was found, as in the following plot.

```julia
%%julia
errs = @.  [end] -  [1:end-1]
plot(0:58,abs.(errs),m=:o,title="Convergence of power iteration",
    xlabel="k",yaxis=("|beta_{60}-beta_k|",:log10,[1e-10,1]))
```



The results are very similar until the last few iterations, when the limited accuracy of the reference value begins to show. That is, while it is a good estimate of λ_1, it is less good as an estimate of the error in nearby estimates.

The practical utility of our calculated convergence rate for the Power Iteration is limited because if we knew λ_1 and λ_2, we wouldn't be running the power iteration in the first place!

Sometimes it's possible to find estimates of or bounds on the ratio. If nothing else, though, it is useful to know that linear convergence is expected at a rate based solely on the dominant eigenvalues.

2 Inverse iteration

Power iteration finds only the dominant eigenvalue. One can employ *deflation* techniques to alter the matrix $\mathbf{A}$ to find the next largest eigenvalue, but this is not always practical.

We next show that Power Iteration can be adapted to find any eigenvalue, provided you start with a reasonably good estimate of it. Some simple linear algebra is all that is needed.

2.1 Inverse iteration

Theorem:

Let $\mathbf{A}$ be an $n \times n$ matrix with eigenvalues $\lambda_1, \ldots, \lambda_n$ (possibly with repeats), and let s be a complex scalar. Then:

1. The eigenvalues of the matrix $\mathbf{A} - s\mathbf{I}$ are $\lambda_1 - s, \ldots, \lambda_n - s$.
2. If s is not an eigenvalue of $\mathbf{A}$, the eigenvalues of the matrix $(\mathbf{A} - s\mathbf{I})^{-1}$ are $(\lambda_1 - s)^{-1}, \ldots, (\lambda_n - s)^{-1}$.
3. The eigenvectors associated with the eigenvalues in the first two parts are the same as those of $\mathbf{A}$.

Proof:

The equation $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ implies that $(\mathbf{A} - s\mathbf{I})\mathbf{v} = \mathbf{A}\mathbf{v} - s\mathbf{I}\mathbf{v} = \lambda\mathbf{v} - s\mathbf{v} = (\lambda - s)\mathbf{v}$. That proves the first part of the theorem.

For the second part, we note that by assumption, $(\mathbf{A} - s\mathbf{I})$ is nonsingular, so $(\mathbf{A} - s\mathbf{I})\mathbf{v} = (\lambda - s)\mathbf{v}$ implies that $\mathbf{v} = (\lambda - s)^{-1}(\mathbf{A} - s\mathbf{I})\mathbf{v}$, or $(\lambda - s)^{-1}\mathbf{v} = (\mathbf{A} - s\mathbf{I})^{-1}\mathbf{v}$.

The discussion above also proves the third part of the theorem.

Consider first part 2 of the theorem with $s = 0$, and suppose that $\mathbf{A}$ has a *smallest* eigenvalue,

$$|\lambda_n| \geq |\lambda_{n-1}| \geq \cdots > |\lambda_1|.$$

Then $|\lambda_1^{-1}| > |\lambda_2^{-1}| \geq \cdots \geq |\lambda_n^{-1}|$, and $\mathbf{A}^{-1}$ has a dominant eigenvalue.

Hence power iteration on $\mathbf{A}^{-1}$ can be used to find the eigenvalue of $\mathbf{A}$ closest to zero.

For nonzero values of s, then we suppose there is an ordering

$$|\lambda_n - s| \geq \cdots \geq |\lambda_2 - s| > |\lambda_1 - s|.$$

Then it follows that

$$|\lambda_1 - s|^{-1} > |\lambda_2 - s|^{-1} \geq \cdots \geq |\lambda_n - s|^{-1},$$

and power iteration on the matrix $(\mathbf{A} - s\mathbf{I})^{-1}$ converges to $(\lambda_1 - s)^{-1}$, which is easily solved for λ_1 itself.

2.2 Algorithm

A literal application of the Power Iteration would include the step

$$\mathbf{y}_k = (\mathbf{A} - s\mathbf{I})^{-1}\mathbf{x}_k.$$

As always, we do not want to explicitly find the inverse of a matrix. Instead we should write this step as the solution of a linear system.

Algorithm: Inverse iteration

Given matrix $\mathbf{A}$ and shift s:

1. Choose $\mathbf{x}_1$.
2. For $k = 1, 2, \ldots,$

a. Solve for $\mathbf{y}_k$ in

$$(\mathbf{A} - s\mathbf{I})\mathbf{y}_k = \mathbf{x}_k.$$

b. Find m such that $|y_{k,m}| = \|\mathbf{y}_k\|_\infty$.

c. Set $\alpha_k = \dfrac{1}{y_{k,m}}$ and $\beta_k = s + \dfrac{x_{k,m}}{y_{k,m}}$.

d. Set $\mathbf{x}_{k+1} = \alpha_k \mathbf{y}_k$.

In the Power Iteration, we used $y_{k,m}/x_{k,m}$ as an estimate of the dominant eigenvalue of $\mathbf{A}$. Here, that ratio is an estimate of $(\lambda_1 - s)^{-1}$, and solving for λ_1 gives the β_k in the Inverse Iteration.

Each pass of inverse iteration requires the solution of a linear system of equations with the matrix $\mathbf{B} = \mathbf{A} - s\mathbf{I}$. This solution might use methods we consider later in this course. Here, we use (sparse) PLU factorization and hope for the best. Since the matrix $\mathbf{B}$ is constant, the factorization needs to be done only once for all iterations.

```
%%julia
"""
    inviter(A,s,numiter)

Perform `numiter` inverse iterations with the matrix `A` and shift
`s`, starting from a random vector. Returns a vector of
eigenvalue estimates and the final eigenvector approximation.
"""
function inviter(A,s,numiter)
    n = size(A,1)
    x = normalize(randn(n),Inf)
      = zeros(numiter)
    fact = lu(A - s*I)
    for k in 1:numiter
        y = fact\x
        normy,m = findmax(abs.(y))
         [k] = x[m]/y[m] + s
        x = y/y[m]
    end
    return  ,x
end
```

```
inviter (generic function with 1 method)
```

2.3 Convergence

The convergence is linear, at a rate found by reinterpreting the Power Iteration convergence rate with $(\mathbf{A} - s\mathbf{I})^{-1}$ in place of $\mathbf{A}$:

$$\frac{\beta_{k+1} - \lambda_1}{\beta_k - \lambda_1} \to \frac{\lambda_1 - s}{\lambda_2 - s} \quad \text{as} \quad k \to \infty,$$

with the eigenvalue order. Thus, the convergence is best when the shift s is close to the target eigenvalue λ_1, specifically when it is much closer to that eigenvalue than to any other.

```
%%julia
  = [1,-0.75,0.6,-0.4,0]
```

```
# Make a triangular matrix with eigenvalues on the diagonal.
A = triu(ones(5,5),1) + diagm( )
```

```
5×5 Matrix{Float64}:
 1.0   1.0   1.0   1.0  1.0
 0.0  -0.75  1.0   1.0  1.0
 0.0   0.0   0.6   1.0  1.0
 0.0   0.0   0.0  -0.4  1.0
 0.0   0.0   0.0   0.0  0.0
```

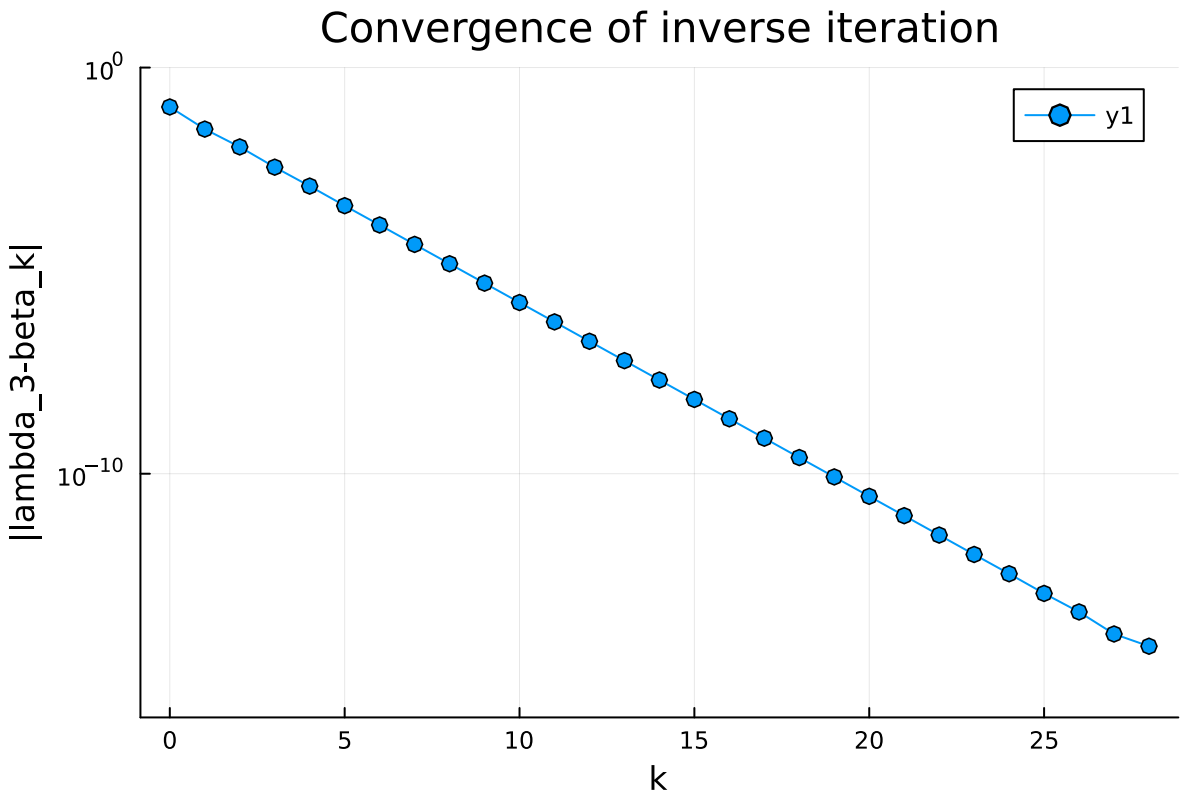
As expected, Inverse Iteration finds the eigenvalue closest to 0.7.

```
%%julia
s = 0.7
 ,x = inviter(A,s,30)
eigval =  [end]
```

```
0.5999999999999985
```

The convergence is again linear.

```
%%julia
errs = @. abs(eigval- )
plot(0:28,errs[1:end-1],m=:o,
    title="Convergence of inverse iteration",
    xlabel="k",yaxis=("|lambda_3-beta_k|",:log10,[1e-16,1]))
```



The observed linear convergence rate is found from the data.

```
%%julia
@show observed_rate = errs[22]/errs[21];
```

```
observed_rate = errs[22] / errs[21] = 0.33326274149401636
```

We reorder the eigenvalues to sort according to the shifted magnitude.

```
%%julia
  =  [ sortperm(abs.( .-s)) ]
```

```
5-element Vector{Float64}:
  0.6
  1.0
  0.0
 -0.4
 -0.75
```

Hence the theoretical convergence rate is

```
%%julia
@show theoretical_rate = ( [1]-s) / ( [2]-s);
```

```
theoretical_rate = ( [1] - s) / ( [2] - s) = -0.3333333333333332
```

2.4 Dynamic shifting

There is a clear opportunity for positive feedback in the Inverse Iteration method. The convergence rate of inverse iteration improves as the shift gets closer to the true eigenvalue—and the algorithm computes improving eigenvalue estimates! If we update the shift to $s = \beta_k$ after each iteration, the convergence accelerates.

Let's analyze the resulting convergence. If the eigenvalues are ordered by distance to s, then the convergence is linear with rate $|\lambda_1 - s|/|\lambda_2 - s|$. As $s \to \lambda_1$, the change in the denominator is negligible. So if the error $(\lambda_1 - s)$ is ϵ, then the error in the next estimate is reduced by a factor $O(\epsilon)$.

That is, ϵ becomes $O(\epsilon^2)$, which is *quadratic* convergence.

```
%%julia
  = [1,-0.75,0.6,-0.4,0]
# Make a triangular matrix with eigenvalues on the diagonal.
A = triu(ones(5,5),1) + diagm( )
```

```
5×5 Matrix{Float64}:
 1.0   1.0   1.0   1.0  1.0
 0.0  -0.75  1.0   1.0  1.0
 0.0   0.0   0.6   1.0  1.0
 0.0   0.0   0.0  -0.4  1.0
 0.0   0.0   0.0   0.0  0.0
```

We begin with a shift $s = 0.7$, which is closest to the eigenvalue 0.6.

```
%%julia
s = 0.7
x = ones(5)
y = (A-s*I)\x
  = x[1]/y[1] + s
```

```
for k in 1:4
    s =
    x = y/y[1]
    y = (A-s*I)\x
    @show   = x[1]/y[1] + s
end                              # the number of accurate digits is doubling!
```

```
  = x[1] / y[1] + s = 0.5612761406172997
  = x[1] / y[1] + s = 0.5964312884753865
  = x[1] / y[1] + s = 0.5999717091820104
  = x[1] / y[1] + s = 0.5999999978556353
```

There is a price to pay for this improvement. The matrix of the linear system to be solved, $(\mathbf{A} - s\mathbf{I})$, now changes with each iteration. That means that we can no longer do just one LU factorization for the entire iteration. The speedup in convergence usually makes this tradeoff worthwhile, however.

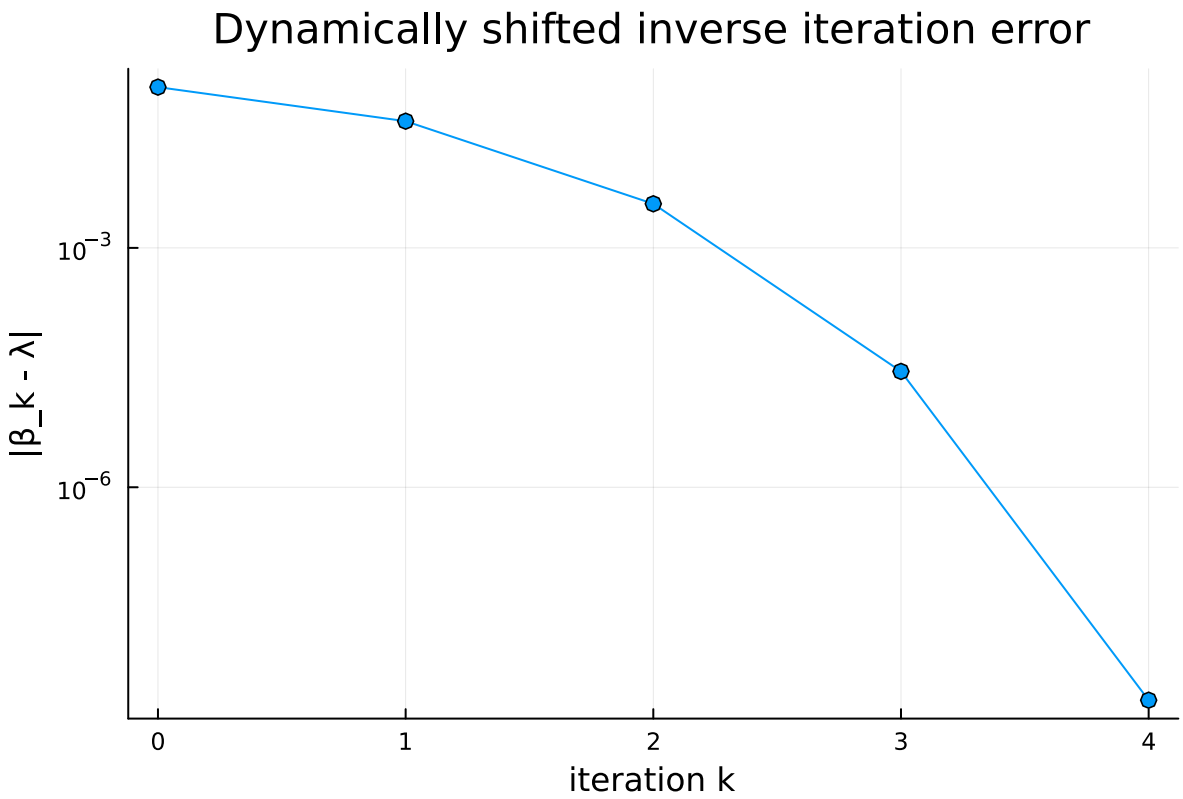
```
%%julia
nmax = 4                        # number of iterations
s = 0.7
x = ones(5)
y = (A - s*I) \ x
  = zeros(nmax + 1)
[1] = x[1]/y[1] + s

for k in 1:nmax
    s =  [k]
    x = y / y[1]
    y = (A - s*I) \ x
     [k+1] = x[1]/y[1] + s
end

# Reference eigenvalue: the exact eigenvalue closest to the final estimate
  = eigvals(A)
star =  [argmin(abs.(  .-  [end]))]

# Error at each iteration (floored at eps so zeros can appear on a log axis)
err = max.(abs.(  .-  star), eps())

plot(0:nmax, err,
     yscale = :log10, marker = :o,
     xlabel = "iteration k", ylabel = "| _k -  |",
     title = "Dynamically shifted inverse iteration error",
     legend = false)
```



In practice power and inverse iteration are not as effective as the algorithms used by `eigs` and based on the mathematics described in the next lectures.